Optimal Control Strategies for 2D Occlusion Aware Dynamic Tracking

Chen-Yu Chang, Gabriel Oliveira, and Hyunchang Kweon

Abstract

This paper explores optimal control strategies for two-dimensional, occlusion-aware dynamic tracking, in which an autonomous quadcopter follows a moving target while avoiding obstacles that block its line of sight (LOS). Three control approaches are developed and assessed in simulation: a real-time Sequential Least Squares Programming Model Predictive Control (SLSQP-MPC) framework with three target trajectory predictors, a Linear Quadratic Regulator (LQR)-based controller incorporating topology-aware LOS-loss recovery and side-bias guidance, and a fully offline optimal control benchmark based on Pontryagin's Maximum Principle (PMP). Simulation results indicate that selecting a trajectory prediction method confers distinct advantages depending on the scenario. The LQR-based approach balances trajectory smoothness and consistent visibility maintenance, whereas the offline PMP solution serves as a reference benchmark.

I. Introduction

A. Occlusion Aware Dynamic Tracking

In a dynamic tracking navigation problem, an agent automatically tracks and follows a chosen target such that it is always centered in its view. Such algorithms are highly applicable to mobile platforms like multi-copter drones for use in cinematography. However, simple control algorithms face the difficulty of consistently tracking a subject due to the existence of obstacles that can obstruct the drone's view, especially when the target tries to navigate around the obstacle. In such a case, the line of sight (LOS) connecting the drone to the target can be intersected by an obstacle in the way. We define such a case as occlusion.

This paper aims to explore multiple real-time approaches in which a drone considers both the position of the target and obstacle to output a trajectory in which occlusion can be effectively avoided while minimizing control efforts. The paper will also present an offline approach by comparing the optimal control strategy from Pontryagin/ Hamiltonian with the MPC control method. Please check out the respository at: https://github.com/tkweon426/Occlusion_Tracking_2D

B. Dynamics Formulation

We model a scenario where the target is a human entity and the drone is a commercially available quadcopter. For computational efficiency and simplification of the problem, we assume that the drone travels at the same altitude, and hence control in the z-axis is neglected. This gives:

$$\mathbf{x} = \begin{bmatrix} x, & y, & \psi, & \dot{x}, & \dot{y}, & \dot{\psi} \end{bmatrix}^\top, \quad \mathbf{u} = \begin{bmatrix} \phi, & \theta, & \tau_z \end{bmatrix}^\top \tag{1}$$

x, y: Position, ψ: Yaw angle, ϕ, θ: Roll and Pitch control angles, τ_z: Yaw torque

The total thrust of the motors is not a state or control input as it will be calculated based on the current drone's mass, pitch, and yaw angles to balance weight: $\frac{T}{m} cos(\phi) cos(\theta) = g$

Therefore, the equation of motion of the 2D quadcopter follows the state form below:

$$\underline{\dot{x}} = \begin{bmatrix} \dot{x} \\ \dot{y} \\ \dot{\psi} \\ \ddot{x} \\ \ddot{y} \\ \ddot{\psi} \end{bmatrix} = \begin{bmatrix} x_4 \\ x_5 \\ x_6 \\ g(\tan\theta\cos\psi - \tan\phi\sin\psi) \\ g(\tan\theta\sin\psi + \tan\phi\cos\psi) \\ \frac{1}{I_{zz}}\tau_z \end{bmatrix} \tag{2}$$

The authors are with Aeronautics and Astronautics Engineering, Purdue university, West Lafayette, IN, 47907 USA (e-mail: chan1239@purdue.edu, ggolivei@purdue.edu, kweonh@purdue.edu)

For the application of designing the controllers, we simplify such states using virtual inputs.
Setting $\ddot{x}$ and $\ddot{y}$ as virtual inputs,

$$u_x = \ddot{x} \qquad u_y = \ddot{y} \tag{3}$$

Position states become:

$$z = \begin{bmatrix} x \\ y \\ \dot{x} \\ \dot{y} \end{bmatrix} \qquad \dot{z} = \begin{bmatrix} 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \end{bmatrix} z + \begin{bmatrix} 0 & 0 \\ 0 & 0 \\ 1 & 0 \\ 0 & 1 \end{bmatrix} \begin{bmatrix} u_x \\ u_y \end{bmatrix} \tag{4}$$

The resulting conversion to low level control inputs become:

$$\tan \begin{bmatrix} \theta \\ \phi \end{bmatrix} = \frac{1}{g} \begin{bmatrix} \cos\psi & \sin\psi \\ -\sin\psi & \cos\psi \end{bmatrix} \begin{bmatrix} u_x \\ u_y \end{bmatrix} \tag{5}$$

C. Environment Setup

The simulation environment is implemented as a lightweight PyGame simulation. This allows for real-time video simulations as well as easy integration of recording and logging features that are implemented in the repository. The repository is fully modular, as arguments of environment (obstacle layout), target entity trajectory, and drone controller can be varied part by part.

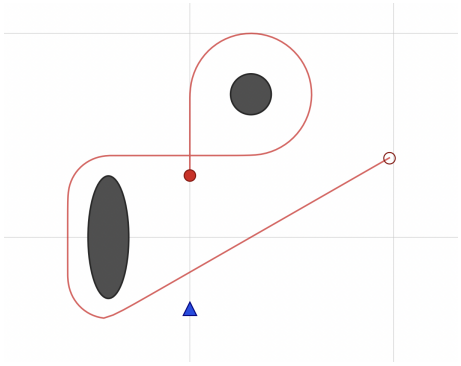


Fig. 1: Simulation Environment and Exemplar Trajectory

For this paper, we use the environment illustrated in figure 1, which contains one circular obstacle and one elliptical obstacle. The target (human entity) is represented as a red dot and the drone is represented as a blue arrow that also shows yaw direction. The dotted line connecting the two subjects shows the line of sight. Each grid in the simulation represents 10 m.

In our setup, we model the speed of the target entity as a fast human jogger with speed = 2.68 m/s [2]. For the drone, we reference the physical specification sheet of the DJI Mavic 3 model [3] which states a max tilt angle of 35°. Since the drone maintains level flight, our acceleration in pitch and roll is clipped at 6.87 m/s^2.

$$9.81 \cdot \tan(35^\circ) = 6.87 m/s^2 \tag{6}$$

II. SLSQP MPC Controller with Trajectory Predictor Exploration

A. State to Control Mapping

In this approach, we attempt to reduce the optimization vector to only the control inputs. This allows the MPC optimization formulation to be simplified to planar accelerations.

$$\mathbf{u} = [a_{x,0}, a_{y,0}, \ldots, a_{x,N-1}, a_{y,N-1}]^T \in \mathbb{R}^{2N} \tag{7}$$

The position x_k at a future step k is calculated with the sequence of accelerations up to that point.

$$x_k = x_0 + k\Delta t v_{x0} + \sum_{i=0}^{k-1} \Delta t^2 (k-i) a_{x,i} \tag{8}$$

which gives us the predicted trajectory vector $\mathbf{x} = [x_1, x_2, \ldots, x_N]^T$.
We split the state vector $\mathbf{x}$ into an unforced response and forced response part by control input

$$\mathbf{x} = \mathbf{x}_{\text{unf}} + M\mathbf{a}_x \tag{9}$$

such that M can be represented in the lower triangular form:

$$M = \Delta t^2 \begin{bmatrix} 1 & 0 & 0 & \ldots & 0 \\ 2 & 1 & 0 & \ldots & 0 \\ 3 & 2 & 1 & \ldots & 0 \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ N & N-1 & N-2 & \ldots & 1 \end{bmatrix} \tag{10}$$

This allows us to calculate the gradient of the cost function J with respect to the input a_x by:

$$\frac{\partial J}{\partial \mathbf{a}_x} = \left(\frac{\partial \mathbf{x}}{\partial \mathbf{a}_x}\right)^T \frac{\partial J}{\partial \mathbf{x}} \quad \rightarrow \quad \nabla_{\mathbf{a}_x} J = M^T \nabla_{\mathbf{x}} J \tag{11}$$

where from equation 9 is obvious that $\frac{\partial \mathbf{x}}{\partial \mathbf{a}_x} = M$.

B. Cost Function Formulation with Analytical Gradients

We formulate the cost function by considering the aspects of control effort, tracking distance, and occlusion. In each section, we introduce the cost function and calculate their respective analytical gradients.

$$J(\mathbf{u}) = J_{\text{eff}} + J_{\text{dist}} + J_{\text{occ}} \tag{12}$$

1) Control Effort Cost: The control effort J_{eff} is calculated as the L_2 norm of the entire control vector, which guides the generated trajectory to result in a smoother flight path.

$$J_{\text{eff}} = w_{\text{eff}} \|\mathbf{u}\|^2 = w_{\text{eff}} \mathbf{u}^T \mathbf{u} \quad \rightarrow \quad \nabla_{\mathbf{u}} J_{\text{eff}} = 2 w_{\text{eff}} \mathbf{u} \tag{13}$$

2) Distance Tracking Cost: The Euclidean distance d_k between the predicted drone position and predicted evader position is taken at every future time step k. This introduces a quadratic cost J_{dist} for the drone to maintain a target distance d_{target}.

$$J_{\text{dist}} = w_{\text{dist}} \sum_{k=1}^{N} (d_k - d_{\text{target}})^2 \tag{14}$$

We first find the gradient with respect to the spatial coordinates by applying the chain rule $\frac{\partial J}{\partial d} \frac{\partial d}{\partial \mathbf{x}}$. The detailed derivation can be found in Appendix A. Then equation 11 gives the gradient with respect to the input.

$$\frac{\partial J_{\text{dist}}}{\partial \mathbf{x}} = 2 w_{\text{dist}} \left(1 - \frac{d_{\text{target}}}{\mathbf{d}}\right) (\mathbf{x} - \mathbf{x_e}) \quad \rightarrow \quad \nabla_{\mathbf{u}} J_{\text{dist}} = M^T \frac{\partial J_{\text{dist}}}{\partial \mathbf{x}, \mathbf{y}} \tag{15}$$

,

3) Occlusion Cost: In this approach, we use the perpendicular distance from the center of the obstacle to the LOS (which we refer to as the LOS clearance distance) as a cost variable, referencing from [4]. In the setting proposed in this paper, the obstacles are either elliptical or circular in shape. To facilitate the computation related to ellipses, we apply a warp transformation relative to an elliptical obstacle with a center (c_x, c_y), a rotation θ, and radii r_x, r_y.

$$u_D = \frac{(x - c_x)\cos\theta + (y - c_y)\sin\theta}{r_x} \qquad v_D = \frac{-(x - c_x)\sin\theta + (y - c_y)\cos\theta}{r_y} \tag{16}$$

This models the elliptical obstacle as a perfectly uniform unit circle placed on $(0, 0)$ in the new coordinate system (u, v). The drone's position is given by (u_D, v_D) and the position of the target is given as (u_E, v_E). The LOS clearance distance is then given by:

$$d = \frac{|S|}{N_{\text{ray}}} = \frac{|u_D v_E - v_D u_E|}{\sqrt{(u_E - u_D)^2 + (v_E - v_D)^2}} \tag{17}$$

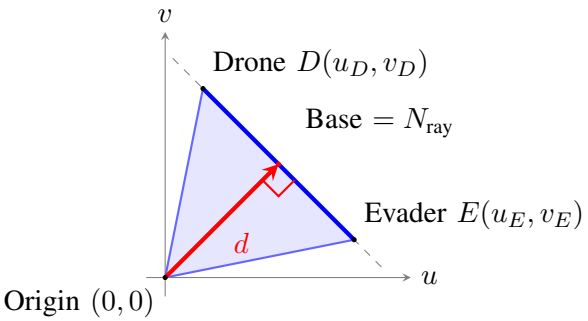


Fig. 2: Geometric representation of the 2D point-to-line distance

Using this distance as the variable, we refer to the exponential distance cost introduced in the work of Messikomer et al [6], and formulate the cost as:

$$J_{\text{occ}} = \sum w_{\text{occ}} \exp(-\alpha(d - 1.0)) \tag{18}$$

This creates an artificial potential field (APF) around the center of the obstacle, such that the LOS is repelled away from the obstacle, with the cost increasing exponentially the closer it is to the center of the object. Note that α is the parameter that controls the steepness of this cost.

Noting the expression $P = w_{\text{occ}} \exp(-\alpha(d - 1.0))$, we find the analytical gradient $\frac{\partial P}{\partial \mathbf{x}}$ given by the chain rule of the variables in equations 16 through 18.

which through an expansive derivation shown in Appendix B comes to be:

$$\frac{\partial P_k}{\partial x_k} = \frac{-\alpha P_k}{N_{\text{ray},k}^3}\left[\text{sgn}(S_k) N_{\text{ray},k}^2 \underbrace{\left(v_{E,k}\frac{\cos\theta}{r_x} + u_{E,k}\frac{\sin\theta}{r_y}\right)}_{\text{from } \frac{\partial S}{\partial x}} + |S_k| \underbrace{\left((u_{E,k} - u_{D,k})\frac{\cos\theta}{r_x} - (v_{E,k} - v_{D,k})\frac{\sin\theta}{r_y}\right)}_{\text{from } -0.5\frac{\partial N^2}{\partial x}}\right] \tag{19}$$

These spatial gradients can then be turned into expressions with respect to the control input a_x by:

$$\frac{\partial J_{\text{occ}}}{\partial a_{x,i}} = \Delta t^2 \sum_{k=i+1}^{N} (k - i)\frac{\partial P_k}{\partial x_k} \tag{20}$$

C. Safety Constraints

In this section, the safety constraints are introduced, and their Jacobians with respect to spatial positions are derived. Owing to equation 9, the spatial constraint gradients can be converted into the control Jacobian by:

$$\frac{\partial C_k}{\partial \mathbf{a}_x} = \left(\frac{\partial C_k}{\partial x_k}\right) M_{k,:} \tag{21}$$

1) Minimum Distance Constraint: To adhere to the minimum safety distance constraints, the following constraint is satisfied at every single time step k in the predicted horizon. Using similar derivations from equation 15, the Jacobian of this constraint term is also calculated.

$$C_{\text{min},k} = d_k - d_{\text{min}} \geq 0 \quad \forall k \in \{1, \ldots, N\} \quad \rightarrow \quad \frac{\partial C_{\text{min},k}}{\partial x_k} = \frac{x_k - x_{e,k}}{d_k} \tag{22}$$

2) Maximum Distance Constraint: In order to keep the drone in an appropriate distance for CV modules to work effectively, we restrict the terminal step position of the drone in the MPC horizon to be constrained by the following. The Jacobian is derived in the same manner as Equation 22.

$$C_{\text{max},N} = d_{\text{max}} - d_N \geq 0 \quad \rightarrow \quad \frac{\partial C_{\text{max},N}}{\partial x_N} = -\frac{x_N - x_{e,N}}{d_N} \tag{23}$$

3) Obstacle Collision Avoidance: A hard collision constraint is applied to stop drones from crashing into obstacles. We reuse the coordinate transformation applied in the occlusion constraint from equation 16. To keep the drone outside the bounds of the obstacle, the constraint is formulated as:

$$C_{\text{obs},k} = \sqrt{l_{x,k}^2 + l_{y,k}^2} - 1.0 \geq 0 \tag{24}$$

The obstacle constraint Jacobian can be derived as:

$$\frac{\partial C_{\text{obs},k}}{\partial x_k} = \frac{1}{2\sqrt{l_x^2 + l_y^2}} \frac{\partial}{\partial x_k}(l_x^2 + l_y^2) = \frac{1}{2\sqrt{l_x^2 + l_y^2}} \left(2l_x \frac{\partial l_x}{\partial x_k} + 2l_y \frac{\partial l_y}{\partial x_k} \right) = \frac{l_x \frac{\partial l_x}{\partial x_k} + l_y \frac{\partial l_y}{\partial x_k}}{\sqrt{l_x^2 + l_y^2}} \tag{25}$$

where the base derivatives are given as:

$$\frac{\partial l_x}{\partial x_k} = \frac{\cos\theta}{r_x} \qquad \frac{\partial l_y}{\partial x_k} = \frac{-\sin\theta}{r_y} \tag{26}$$

D. Predictors Design for the MPC Horizon

1) Constant Velocity Trajectory Predictor: This method uses constant velocity assumption to roll out the MPC horizon trajectory of the evader. Upon receiving a new position measurement $\mathbf{p}_t$, an instantaneous velocity vector $\tilde{\mathbf{v}}_t$ is computed using a backward finite difference with respect to the previous observation $\mathbf{p}_{t-1}$:

$$\tilde{\mathbf{v}}_t = \frac{\mathbf{p}_t - \mathbf{p}_{t-1}}{\Delta t_{\text{sim}}} \tag{27}$$

2) Velocity & Acceleration based Trajectory Predictor: This method uses the position measurements to calculate both the velocity and acceleration. It also applies a exponential moving average (EMA) for smoothing. The velocity is calculated in the same manner as equation 27. The EMA is applied by:

$$v_k = \alpha_v v_{\text{raw},k} + (1 - \alpha_v) v_{k-1} \tag{28}$$

acceleration is calculated in the same manner:

$$a_{\text{raw},k} = \frac{v_k - v_{k-1}}{\Delta t_{\text{sim}}} \tag{29}$$

$$a_k = \alpha_a a_{\text{clipped},k} + (1 - \alpha_a) a_{k-1} \tag{30}$$

3) Attractive Field based Trajectory Predictor: We propose a new formulation of a trajectory predictor. We pose the target as an active evader that tries to maximize occlusion, but at a cheap computational cost. This is achieved by introducing a gravity-like heuristic to the obstacles, such that it applies an attractive acceleration to the future horizon states of the target. This causes the future trajectory of the target to wrap around the obstacle.

We first calculate the base velocity by finite differencing as explained in equation 27. The position is then updated by combining the base velocity with the attractive velocity v_{att}.

$$p_{k+1} = p_k + \Delta t(v_{\text{base}} + v_{\text{att},k}) \tag{31}$$

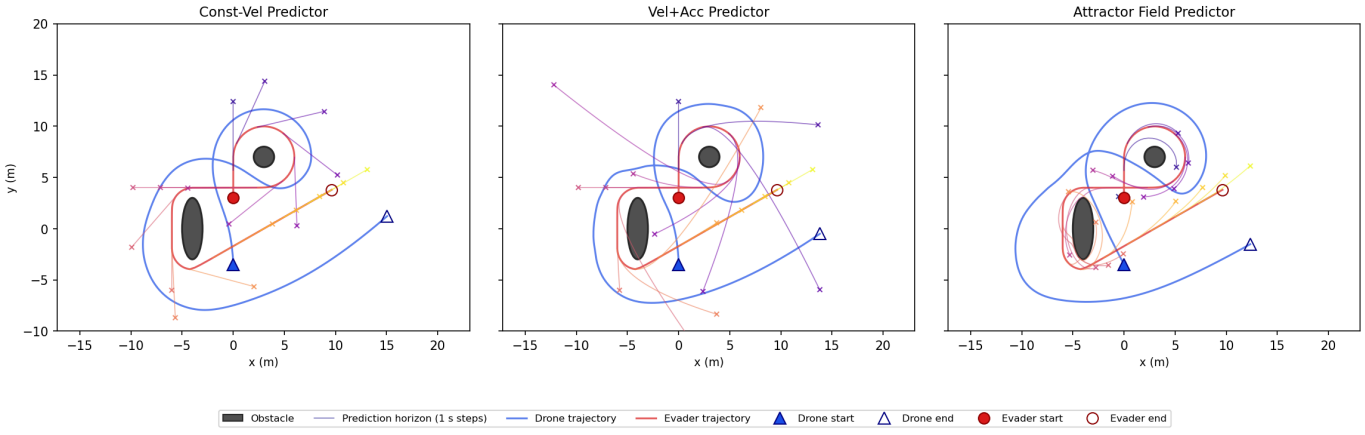
Then the accumulated attractive velocity is updated for the next step as:

$$v_{\text{att},k+1} = v_{\text{att},k} + \Delta t \sum_j a_{\text{att}}(p_{k+1}, O_j) \tag{32}$$

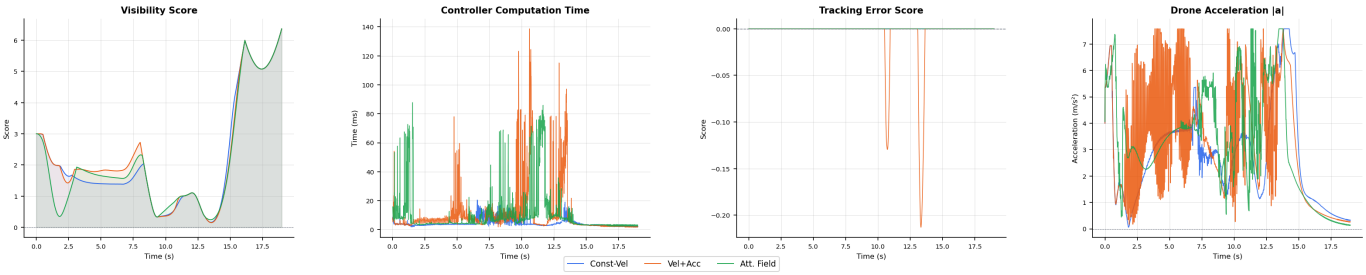
Acceleration is calculated with an inverse-square relation to distance-like gravity. It is also capped at the maximum acceleration of the drone itself.

$$a_{\text{mag}} = \min\left(\frac{k_{\text{att}}}{d_{\text{surf}}^2},\ a_{max,drone}\right) \tag{33}$$

E. Results



(a) Visual Trajectory



(b) Trajectory Metrics

Fig. 3: Trajectory visualization and accompanying metrics.

The results from figure 3 show that the constant velocity predictor produces the smoothest trajectory. The velocity & acceleration predictor produces the roughtest trajectory. This is supported by the numbers shown in table II, by the acceleration costs.

Each type of a predictor seems to have a unique strength: the constant velocity predictor is good at minimizing acceleration costs, while the velocity & acceleration predictor had advantages in maximizing the average visibility score. While the attractive field predictor has a low average visibility score, it actually has the best metric for minimum visibility score. This is inline with the intent of the design, as we intended it to perform like an active evader that tries to maximize occlusion. The controller moves to minize this behavior, resulting in the highest

minimum visibility score. Note that the results were obtained with fixed controller weights, and an extensive ablation study still remains to assess the best case results of each combination. I would also like to further explore the attractive field model for a more mathematically intricate method to mimic an active evader.

		Constant Velocity	Velocity & Acceleration	Attractive Field
Visibility Score	**Average**	2.131	2.223	2.079
	Min	0.160	0.148	0.243
Tracking Error	**Average**	0.000	-0.006	0.000
	Min	0.000	-0.213	0.000
Controller Computation (ms)	**Average**	4.602	10.114	9.888
	Max	20.573	138.687	87.815
	Min	1.704	1.677	2.316
Acceleration	**Acceleration Cost (m^2/s^4)**	11.450	16.075	14.362
	Min (m/s^2)	0.061	0.169	0.124

TABLE I: Summary of metrics

III. LQR Based Controller

Based on the environment setup above, we attempt to make the drone tracking the target while maintaining visibility with the dynamic constraints, considering the optimal costs and constraints from the work of Masnavi et al. [5], it provides an effective framework for occlusion constraints, visibility, and cost terms such as smoothness of the trajectory are insufficient to ensure full tracking behavior.

A. LQR Cost Function

To address this limitation, we introduce an LQR-inspired quadratic cost into the controller. In LQR, a quadratic cost function is used to balance tracking performance and control efficiency; this idea is useful in our problem, since the drone should stay close to the target while avoiding aggressive acceleration. In the original occlusion formula, the cost is mainly focused on the smoothness of the trajectory .

$$J = \sum_{k=0}^{N-1} \left[w_a \|u_k\|^2 + w_t (x_k - x_{d,k})^T Q (x_k - x_{d,k}) \right] + (x_N - x_{d,N})^T Q_f (x_N - x_{d,N}) \tag{34}$$

J: cost, N: horizon length, u_k: control input, x_k: state, $x_{d,k}$: desired state, w_a: control-effort weight, w_t: tracking-error weight, Q: running penalty matrix, Q_f: terminal penalty matrix.

B. Hard Obstacle Constraints

To prevent the agent from colliding the obstacles, we enlarge the obstacles' boundary by a certain buffer, as shown in Eq. (10).

$$\begin{bmatrix} r_{x,\text{safe}} \\ r_{y,\text{safe}} \end{bmatrix} = \begin{bmatrix} r_x + b_{\text{hard}}, \\ = r_y + b_{\text{hard}} \end{bmatrix} \tag{35}$$

Then the agent's position is transformed into the obstacles' coordinate frame, making it easier to write the constraints for the ellipse. If the obstacles rotate, this equation rotates the coordinate system.

$$\begin{bmatrix} q_x \\ q_y \end{bmatrix} = \begin{bmatrix} \cos\theta(x - x_o) + \sin\theta(y - y_o) \\ -\sin\theta(x - x_o) + \cos\theta(y - y_o) \end{bmatrix} \tag{36}$$

This is hard constraint for the ellipse obstacles:

$$g(x, y) = \left(\frac{q_x}{r_{x,\text{safe}}} \right)^2 + \left(\frac{q_y}{r_{y,\text{safe}}} \right)^2 - 1, \qquad g(x, y) \geq 0 \tag{37}$$

Since the true ellipse is nonlinear, so we approximates it by a straight-line tangent constraint around the previous trajectory point p_k^0.

$$\nabla g_k^T p_k \geq \nabla g_k^T p_k^0 - g_k^0 \tag{38}$$

C. Side Bias Visibility

Although the hard constraints avoid collisions, it does not tell the agent which side to pass the obstacle. This can cause unstable behavior. To reduce this problem, a bias-side penalty is added when the agent gets close to the obstacles.

$$d_{\text{obs},k} = \|p_{r,k} - p_{\text{bias obs}}\|, \qquad d_{\text{obs},k} < R_{\text{side}} \tag{39}$$

Eq. (15) checks whether the agent is close enough to the obstacles to activate the side-bias behavior, and then we define the bias weight, shown as Eq. (16).

$$w_{\text{side},k} = W_{\text{side}}\sigma_k^2, \qquad \sigma_k = 1 - \frac{d_{\text{obs},k}}{R_{\text{side}}} \tag{40}$$

This creates a preferred side position beside the target and the agent is encouraged to stay on a chosen side, which is set to be left and down, instead of choosing random direction around the obstacles.

$$p_{\text{side},k} = p_{r,k} + d_{\text{side}}n_{\text{side}}, \qquad n_{\text{side}} = \frac{1}{\sqrt{2}}\begin{bmatrix}-1\\-1\end{bmatrix} \tag{41}$$

The cost function below is not a hard constraint, but a soft guidance term:

$$J_{\text{side}} = \sum_k w_{\text{side},k}\,\|p_k - p_{\text{side},k}\|^2 \tag{42}$$

D. Topology-aware LOS-loss recovery

The side-bias term helps the agent choose a stable side, but it may not be enough when the LOS is already blocked. Therefore, a recovery layer is added to detect visibility loss and bring the agent back to a better position of viewing.

This equation checks how close each LOS point is to the obstacle and tells us whether the LOS is clear or blocked:

$$c_j = (E_j - 1)\min(r_x, r_y), \qquad c_{\min} = \min_{i,j} c_{i,j}, \qquad E_j = \sqrt{\left(\frac{q_x}{r_x}\right)^2 + \left(\frac{q_y}{r_y}\right)^2} \tag{43}$$

After the decision on the activation of recovery, which mainly depends on whether the minimum clearance is below the recovery clearance, we need to find the direction from the obstacle to the evader and then create two possible recovery directions, left or right.

$$r = p_{\text{evader}} - p_{\text{obs}}, \qquad \hat{r} = \frac{r}{\|r\|} \tag{44}$$

$$n_{\text{rec}} = \begin{cases} n_L, & \|p_{\text{agent}} - p_L\| \le \|p_{\text{agent}} - p_R\| \\ n_R, & \text{otherwise} \end{cases}, \quad n_L = \begin{bmatrix}-\hat{r}_y\\ \hat{r}_x\end{bmatrix}, \quad n_R = \begin{bmatrix}\hat{r}_y\\ -\hat{r}_x\end{bmatrix} \tag{45}$$

Then we define recovery points and recovery weight:

$$p_L = p_{\text{evader}} + d_{\text{rec}}n_L, \qquad p_R = p_{\text{evader}} + d_{\text{rec}}n_R. \qquad w_{\text{rec},k} = W_{\text{rec}}(1 + s_{\text{rec}})\gamma_k \tag{46}$$

$$p_{\text{rec},k} = p_{r,k} + d_{\text{rec}}n_{\text{rec}}, \qquad s_{\text{rec}} = \text{clip}\left(\frac{c_{\text{rec}} - c_{\min}}{c_{\text{rec}}}, 0, 2\right), \qquad \gamma_k = \text{linspace}(1.00, 0.10, N) \tag{47}$$

where $p_{\text{rec,k}}$ is the point agent should move toward during recovery,s_{rec} shows how serious the LOS block is, and γ_k makes the recovery effect stronger at the beginning.

$$J_{\text{rec}} = \sum_k w_{\text{rec},k}\,\|p_k - p_{\text{rec},k}\|^2 \tag{48}$$

E. Results

Figure 4 compares the two controllers using four different metrics: visibility score, computation time, tracking error, and acceleration magnitude. The LQR-based optimized controller generally improves visibility, reduces tracking error violation, and makes acceleration less aggressive. This shows that the added penalties and filter improve the tracking smoothness and safety.

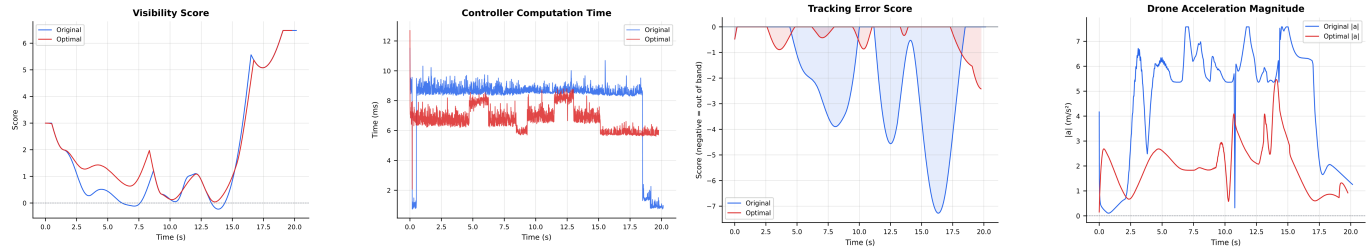


Fig. 4: Comparison of the original and optimal controllers in four evaluation metrics: visibility score, computation time, tracking error score, and acceleration magnitude.

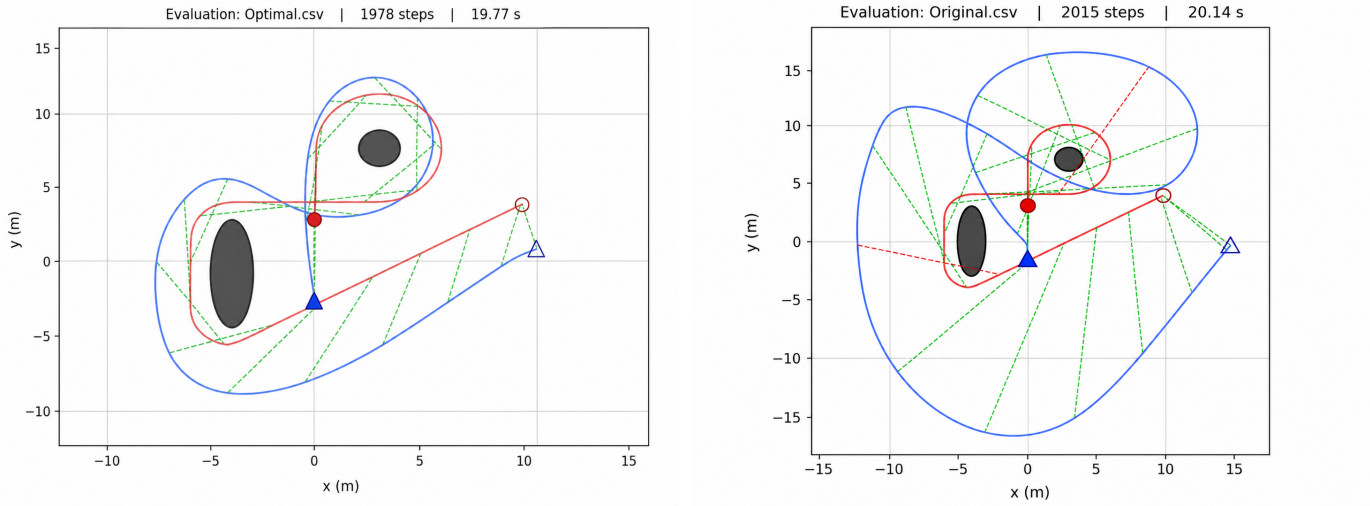


Fig. 5: Comparison of trajectory of the original controller and the optimal controller

Figure 5 shows the trajectory comparison between the original controller and the optimized one, The optimized controller follows a smoother path around the obstacles and keeps the LOS more consistently.

IV. Pontryagin-Based Full-Offline Optimal Control Benchmark

To complement the online MPC controllers, we formulate a full-offline optimal-control benchmark using Pontryagin's Maximum Principle (PMP). Unlike the MPC controllers, which optimize over a short receding horizon, the PMP benchmark uses the complete scripted target trajectory before the simulation begins. The resulting control sequence is computed once over the full scenario and then replayed in the PyGame simulation.

A. Cost Function

The offline benchmark uses the same cost structure as the MPC controllers: control effort, tracking distance, and occlusion avoidance. The discretized cost is

$$J = \sum_{k=0}^{N-1} \left(w_u \|u_k\|_2^2 + w_d (d_k - d_{\text{target}})^2 + w_{\text{occ}} J_{\text{occ},k} + w_{\text{side}} J_{\text{side},k} \right). \tag{49}$$

Here, $J_{\text{u},k}$ penalizes acceleration effort, $J_{\text{d},k}$ penalizes deviation from the desired target distance, $J_{\text{occ},k}$ penalizes line-of-sight configurations that pass near or through an obstacle, and the target-relative distance is defined as

$d_k = \|p_k - p_{e,k}\|_2$. The occlusion term uses the same obstacle-normalized coordinate transformation and LOS distance definition as the SLSQP-MPC formulation in Eqs. (16)–(18).

The only additional soft guidance term in the offline benchmark is the side-bias cost. This term is included to encourage the optimizer to choose a consistent side of the elliptical obstacle when multiple collision-free and LOS-preserving paths are possible. It is written as

$$J_{\text{side},k} = \sigma_k \|p_k - p_{\text{side},k}\|_2^2. \tag{50}$$

The preferred side point is

$$p_{\text{side},k} = p_{e,k} + d_{\text{target}} n_{\text{side}}, \tag{51}$$

where the top-side direction is chosen as

$$n_{\text{side}} = \begin{bmatrix} 0 \\ 1 \end{bmatrix}. \tag{52}$$

The activation weight σ_k is selected to be significant only when the target and drone are near the elliptical obstacle. As a result, the side-bias term does not dominate the entire trajectory, but it helps remove ambiguity around the obstacle.

B. Constraints

The offline PMP benchmark uses the same physical and geometric constraints as the MPC-SLSQP formulation. The acceleration constraint is given by Eq. (49), the target-distance constraints follow the same minimum and maximum distance logic previously introduced and are given by Eq. (22) and (23), the obstacle collision avoidance is also imposed using the same inflated obstacle constraint and is given by Eq. (24). Similarly, line-of-sight preservation is imposed using the same normalized LOS distance $d_{\text{LOS},k}$ defined in Eq. (17).

C. Hamiltonian Formulation

For the continuous-time formulation, the running cost is defined

$$L(z, u, t) = w_u \|u\|_2^2 + w_d (d - d_{\text{target}})^2 + w_{\text{occ}} J_{\text{occ}} + w_{\text{side}} J_{\text{side}}. \tag{53}$$

The Hamiltonian is

$$H(z, u, \lambda, t) = L(z, u, t) + \lambda^T (Az + Bu). \tag{54}$$

Pontryagin's necessary conditions give

$$\dot{z}^*(t) = \frac{\partial H}{\partial \lambda} = Az^*(t) + Bu^*(t), \tag{55}$$

$$\dot{\lambda}^*(t) = -\left(\frac{\partial H}{\partial z}\right)^T, \tag{56}$$

and

$$0 = \frac{\partial H}{\partial u} = 2w_u u^*(t) + B^T \lambda^*(t). \tag{57}$$

Therefore, on an unconstrained arc,

$$u^*(t) = -\frac{1}{2w_u} B^T \lambda^*(t). \tag{58}$$

With the acceleration bound, the implemented control is projected onto the admissible set:

$$u^*(t) = \Pi_{\mathcal{U}} \left(-\frac{1}{2w_u} B^T \lambda^*(t) \right). \tag{59}$$

where $\Pi_{\mathcal{U}}$ works as a saturation block.

D. Offline Numerical Solution

The full problem is solved offline over the complete scripted scenario. The control sequence

$$U = \begin{bmatrix} u_0 & u_1 & \cdots & u_{N-1} \end{bmatrix} \tag{60}$$

is optimized subject to the dynamics and constraints above. The state trajectory is generated using

$$p_{k+1} = p_k + v_k \Delta t + \frac{1}{2} u_k \Delta t^2, \tag{61}$$

$$v_{k+1} = v_k + u_k \Delta t. \tag{62}$$

The resulting trajectory and control sequence are saved and then replayed in the PyGame simulation. Thus, the displayed animation is the visualization of the offline optimized benchmark trajectory.

E. Results

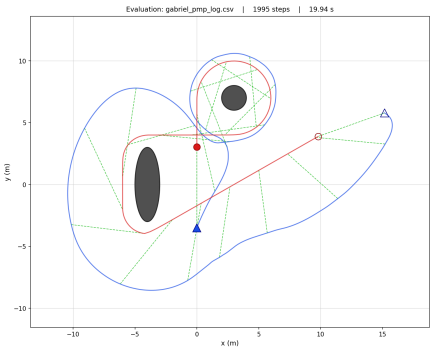


Fig. 6: Offline PMP trajectory generated over the full scripted target path.

		SLSQP-MPC	MPC-LQR	PMP
Visibility Score	**Average**	2.223	1.932	2.060
	Min	0.243	0.042	0.215
Tracking Error	**Average**	0.000	-0.291	0.000
	Min	0.000	-2.420	0.000
Controller Computation (ms)	**Average**	4.602	6.755	N/A
	Max	20.573	12.702	N/A
	Min	1.677	2.062	N/A
Acceleration	**Cost (m^2/s^4)**	11.450	5.011	31.870
	Min (m/s^2)	0.061	0.154	0.166

TABLE II: Summary of controller performance metrics.

V. Conclusion

This report examines several optimal-control strategies for two-dimensional occlusion-aware dynamic tracking. The problem is formulated using reduced planar quadcopter dynamics, requiring the drone to track a moving target while maintaining a specified separation distance, avoiding collisions with obstacles, and preserving line of sight. A lightweight PyGame simulation environment enables comparison of online and offline approaches under identical obstacle configurations, target trajectories, and acceleration constraints.

Results from the SLSQP-MPC controller indicate that trajectory prediction significantly influences tracking and visibility performance. The constant-velocity predictor yielded the smoothest response and the lowest acceleration cost, whereas the velocity-and-acceleration predictor achieved the highest average visibility score. The attractive-field predictor attained the best minimum visibility score, suggesting that modeling the target as a more active evader can effectively test controller robustness in challenging occlusion scenarios.

The LQR-based controller demonstrated that incorporating quadratic tracking penalties, side-bias visibility terms, and line-of-sight recovery logic improves path smoothness and reduces tracking violations compared to the original controller. These enhancements enabled the drone to select more consistent paths around obstacles and maintain visibility with greater reliability.

The PMP-based fully offline benchmark offers a valuable comparison to the online MPC approaches. Because PMP utilizes the complete target trajectory prior to simulation, it serves as a full-information optimal-control benchmark rather than a real-time feedback controller. The PMP trajectory maintained the desired tracking distance and preserved line of sight, but incurred a higher acceleration cost than the online MPC cases. These findings underscore the tradeoff between offline optimal planning, online computational feasibility, tracking accuracy, visibility preservation, and control effort.

Overall, the results demonstrate that occlusion-aware tracking necessitates balancing multiple competing objectives rather than optimizing a single metric. Future research should include a broader ablation study of controller weights, incorporate more realistic target behaviors, and extend the analysis from simplified two-dimensional dynamics to a higher-fidelity quadcopter model.

VI. Contributions of Individual Members

Hyunchang Kweon (Thomas): Formulation of the dynamics, coding of the base environment, section on Occlusion Aware SLSQP MPC Controller with trajectory predictors, Evaluation and Metric calculation scripts.

Chen-Yu Chang: LQR-Based MPC controller with multiple constraints and penalties.

Gabriel Oliveira: Pontryagin-Based Optimal Control Benchmark of full target trajectory

Appendix

A. Proof of the Distance Cost Gradient Term

From the term distance cost, the Euclidean distance d_i between the drone (x_i, y_i) and the evader $x_{e,i}, y_{e,i}$ is the following:

$$d_i = \sqrt{(x_i - x_{e,i})^2 + (y_i - y_{e,i})^2} \tag{63}$$

Taking the partial derivative to the drone's position at a specific step k, then using the chain rule $\frac{\partial J}{\partial x_k} = \frac{\partial J}{\partial d_k} \cdot \frac{\partial d_k}{\partial x_k}$ results in:

$$\frac{\partial J_{\text{dist}}}{\partial x_k} = \frac{\partial}{\partial x_k}\left[w_{\text{dist}}(d_k - d_{\text{target}})^2\right] \quad \rightarrow \quad \frac{\partial J}{\partial d_k} = 2w_{\text{dist}}(d_k - d_{\text{target}}) \tag{64}$$

$$\frac{\partial d_k}{\partial x_k} = \frac{1}{2}\left((x_k - ex_k)^2 + (y_k - ey_k)^2\right)^{-1/2} \cdot 2(x_k - ex_k) = \frac{x_k - ex_k}{d_k} \tag{65}$$

Therefore, the vector gradient over the horizon N becomes:

$$\frac{\partial J_{\text{dist}}}{\partial \mathbf{x}} = 2w_{\text{dist}}\left(1 - \frac{d_{\text{target}}}{\mathbf{d}}\right)(\mathbf{x} - \mathbf{x}_e) \tag{66}$$

B. Proof of the Occlusion cost function Analytical Gradient

Beginning with the occlusion cost term:

$$P \ = \exp\left(-\alpha\left(d - 1.0\right)\right) \tag{67}$$

and recalling that $d = \frac{|S|}{N_{\text{ray}}}$, the quotient rule results in:

$$\frac{\partial d}{\partial x} = \frac{\operatorname{sgn}(S)\,\frac{\partial S}{\partial x} N_{\text{ray}}^2 - 0.5\,|S|\,\frac{\partial N_{\text{ray}}^2}{\partial x}}{N_{\text{ray}}^3} \tag{68}$$

Also recalling that $S = u_D v_E - v_D u_E$ and $N_{\text{ray}}^2 = (u_E - u_D)^2 + (v_E - v_D)^2$ and $\frac{\partial u_D}{\partial x} = \frac{\cos\theta}{r_x} \quad \frac{\partial v_D}{\partial x} = \frac{-\sin\theta}{r_y}$: Differentiating S results in:

$$\frac{\partial S}{\partial x} = \frac{\partial u_D}{\partial x} v_E - \frac{\partial v_D}{\partial x} u_E = v_E\left(\frac{\cos\theta}{r_x}\right) - u_E\left(\frac{-\sin\theta}{r_y}\right) \tag{69}$$

and differentiating N results in:

$$\frac{\partial N_{\text{ray}}^2}{\partial x} - 2\left(u_E - u_D\right)\left(\frac{\cos\theta}{r_x}\right) - 2\left(v_E - v_D\right)\left(\frac{-\sin\theta}{r_y}\right) \tag{70}$$

Then the resulting gradient is found with:

$$\frac{\partial P}{\partial \mathbf{x}} = \frac{\partial P}{\partial d} \cdot \frac{\partial d}{\partial \mathbf{x}} \quad \rightarrow \quad \nabla_{\mathbf{u}} J_{\text{occ}} = M^T \frac{\partial P}{\partial \mathbf{x}, \mathbf{y}} \tag{71}$$

References

[1] Themoonlight.io, 2026, www.themoonlight.io/en/review/differentiable-optimization-based- neural-policy-for-occlusion-aware-target-tracking. Accessed 24 Mar. 2026.

[2] K. Nunez, "What's an Average Jogging Speed?," Healthline, Aug. 24, 2018. https://www.healthline.com/health/average-jogging-speed

[3] "DJI Mavic 3 Classic - Specs - DJI," DJI Official. https://www.dji.com/mavic-3-classic/specs

[4] B. F. Jeon and H. J. Kim, "Online Trajectory Generation of a MAV for Chasing a Moving Target in 3D Dense Environments," Apr. 06, 2019, arXiv: arXiv:1904.03421. doi: 10.48550/arXiv.1904.03421.

[5] H. Masnavi, V. Adajania, K. Kruusamae, and A. K. Singh, "Real-Time Multi-Convex Model Predictive Control for Occlusion Free Target Tracking," Dec. 22, 2021, arXiv: arXiv:2112.12177. doi: 10.48550/arXiv.2112.12177.

[6] N. Messikommer, J. Xing, L. Bauersfeld, M. Cannici, E. Aljalbout, and D. Scaramuzza, "Approximate Imitation Learning for Event-based Quadrotor Flight in Cluttered Environments," Mar. 08, 2026, arXiv: arXiv:2603.07578. doi: 10.48550/arXiv.2603.07578.